PSAI_143

SUNSHINE KEYBASE

- **Proposer:** 4meta5
- **Payment Address:** 16uCivZhPAy4RiK5ZA1TEMo6Xq5pGTh4BA

PROJECT OVERVIEW :PAGE_FACING_UP:

OVERVIEW

sunshine-keybase implements Keybase's Local Key Security to associate a set of keys with a UID and manage this set of keys to sign and authenticate messages on the user's behalf. The implementation uses a Substrate runtime to enforce the rules of interaction with the keystore, especially as it pertains to revoking keys for a provisioned storage device.

The project provides a usable keystore for local first applications using substrate. The sunshine-identity pallet provides structure to control a set of keys with a UID and track storage metadata associated with the given UID. The client uses ipfs-embed to store sensitive data offchain and encrypted on the local hardware. The sunshine-chain pallet uses the same stack to coordinate storage of chain data amongst a closed set of clients.

We believe sunshine-keybase provides a reusable identity architecture for other projects in the Web3 space. Moreover, the sunshine-chain client and pallet demonstrate minimal design to set and enforce the storage of offchain data among a permissioned set of clients. This infrastructure will prove useful to substrate projects that need to coordinate and enforce private network storage of data offchain.

PROJECT DETAILS

The project includes two pallets and a Flutter UI for Android.

Chain Pallet

The chain module is a reusable abstraction for building private proof of authority chains using ipfs and using substrate to provide authorization and consensus on the current head of the chain. When authoring a block on ipfs a race condition can occur. Due to substrate providing a total order of transactions only one transaction will succeed in updating the head of the chain, the other client will create a new block on the head of the chain and retry the failed operation.

chain_module.svg

Keybase Pallet

The keybase identity module manages the user's chain that stores the user key, device keys, password and social media accounts using the sunshine chain pallet. Private data shared between devices is encrypted with the user private key. When a new device is provisioned a key is generated locally on the device, and a provisioning protocol is used to communicate between the new device and the provisioning device.

keybase-module.svg

Password changes are stored encrypted in the user chain. When a device receives a block with a password change it reencrypts it's local device key using the new password. This ensures that the user only needs to remember one password.

Social media accounts are linked to a chain account, by submitting a proof in the social media profile and on the user's chain. Other users can find the on chain account on the social media page and verify that they are both controlled by the same cryptographic identity. This allows us to use github usernames as aliases without compromising the decentralized nature or security that blockchains provide. While resolving the social media account to an on chain identity requires the service to be online, already resolved identities are stored locally. This means that even if github is offline, transfers to already verified github accounts can be performed.

Flutter Android UI

The Flutter UI allows Android users to initialize a local keystore and device identity using the sunshine-identity pallet. It allows the user to initialize the keystore, set a passphrase, and change the passphrase. The UI also allows the user to paste links to github gists to prove ownership of github accounts.

ECOSYSTEM FIT

There are other approaches to creating an identity graph, but we have yet to find another substrate project that implements device-oriented key revocability. The native keystore leverages ipfs-embed to store sensitive data in an encrypted form on the local hardware. Over time, we expect ipfs-embed to evolve with the needs and requirements of users.

The current design coordinates the storage of private offchain data on permissioned client networks. Our project will use this identity infrastructure for sharing encrypted messages and backing up encrypted files, privately and within teams with membership tracked transparently on-chain.

TEAM :BUSTS_IN_SILHOUETTE:

TEAM MEMBERS

- Amar Singh
- David Craven
- Shady Khalifa

TEAM WEBSITE

- https://sunshine.foundation

LEGAL STRUCTURE

Sunshine Systems LLC

16192 Coastal Highway, Lewes, Delaware 19958, County of Sussex

TEAM'S EXPERIENCE

Amar started and maintained the Substrate Recipes as an employee at Parity. He wrote the sunshine-bounty runtime, node, client, and CLI under Sunshine's first grant. He is an experienced substrate runtime developer.

David worked on substrate core development as an employee at Parity. He is the lead maintainer of ipfs-embed and contributes upstream often to substrate-subxt to meet Sunshine's Rust client requirements.

Shady is an experienced Typescript, Flutter, and Rust developer with contributions to many open source projects (i.e. nest-access-control). He is a pioneer in Rust-Dart FFI development with generic components released alongside his work on the Sunshine Flutter infrastructure i.e. flutterust.

TEAM CODE REPOS

- sunshine-protocol
- ipfs-rust
- flutterust

DEVELOPMENT ROADMAP :NUT_AND_BOLT:

There is only one milestone.

MILESTONE 1 — SUNSHINE-KEYBASE + UI

- **Estimated Duration:** 6 weeks
- **FTE:** 2
- **Costs:** 1.5 BTC

Number	Deliverable	Specification
0a.	License	Unlicense
0b.	Documentation	We will provide both inline documentation of the code and a basic tutorial that shows how to run the local dev node alongside the Flutter UI to register and manage a sunshine-identity on a local network.
0c.	Testing Guide	The client will have substrate-subxt integration tests that verify behavior at the network level to ensure functionality and robustness. In the guide, we will describe how to run these tests.
1.	Substrate module: Chain	We will create a pallet that allows a closed set of signers to build private proof of authority chains using ipfs and substrate. These chains consists of private data shared among the closed set.
2.	Substrate module: Identity	We will create a pallet that uses the Chain module to manage data relevant to the registered identity. This module adds more granular management when storing the user key, device keys, password and social media accounts.
3.	Flutter UI	We will write a Flutter UI that communicates directly between our Rust substrate-subxt client and our Flutter interface to express the user identity configuration interface.
4.	Substrate chain	Flutter UI works with local node to enable identity registration, password reset, and github authentication.

5.	Docker	We will provide a dockerfile to demonstrate the full functionality of our chain

COMMUNITY ENGAGEMENT

We will publish an article on dev.to upon completion.

FUTURE PLANS

Sunshine Chain will launch in Q1 2021. We are continuing to build infrastructure that is useful for the interchain community as well as other individual substrate chains.

PAST WORK

We wrote a blog post listing open source contributions funded by our first Web3 grant. Highlights include * 7 governance pallets, configured into a runtime and node to express a Substrate DAO Chain * Rust client and CLI for interacting with the node * substrate-subxt contributions to support native Rust clients and light clients (w/ 2 Parity projects now using substrate-subxt for this purpose — ledgeracio, cargo contract) * Rust client which uses ipfs-embed for storage of data not necessary for Substrate runtime storage * including tooling around ipfs-embed to facilitate error handling and improve the overall API * Rust-Dart FFI to call Rust client code from Flutter Dart User Interface * with intermediate infrastructure open sourced for the wider Rust and Flutter ecosystems * native keystore supports modern key management i.e. key rotation